

P2P Networks

maria.potop-butucaru@lip6.fr

Infrastructures - exemples

- Internet
- P2P Networks – logical infrastructures
 - non-structured
 - structured (Distributed Hash Tables)
- Mobile networks & Sensor networks
 - Clustering – cels, pico-cels
 - Connected Covering
 - backbones

Infrastructures - applications

- Routing and service/data lookup
 - Gnutella, Kazaa, BitTorrent
- Communication and Collaboration
 - Chat/Irc, NewsGroups
 - Instant Messaging (Aol, Icq, Yahoo, Msn)

Infrastructures - applications

- Sharing data, bandwidth, computational power
 - Napster, Publius, Freenet, MojoNation, FreeHaven, Groove, e-donkey, Chord, Can, Pastry, Tapestry, Kademlia
 - Voice/IP (Skype)
 - Seti@home (astronomy)
 - genome@home (ADN)
 - folding@home (proteins)

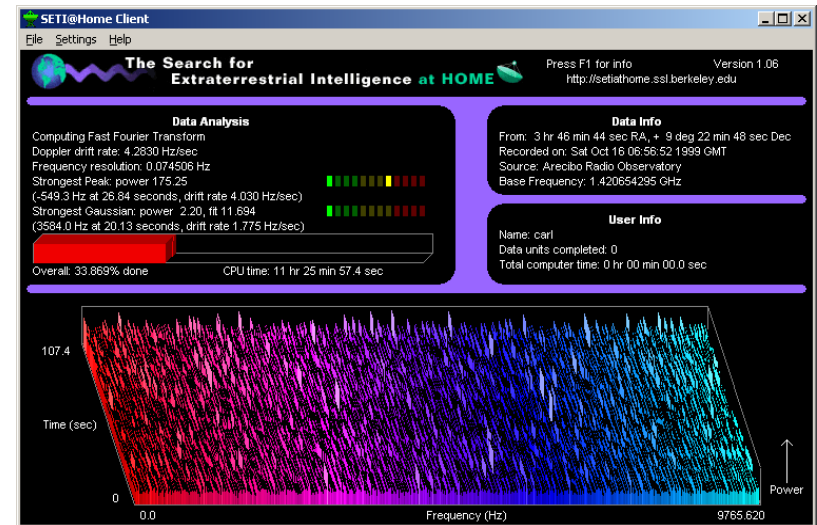


- Looking for the extraterrestrial live
- Computers charge and analyse data collected from the Arecibo telescope.
- 2,4 millions of participants from 226 countries





- 3.8M users from 226 countries
- 1200 years CPU / day
- 1.7 Zettaflop (10^{21}) in the last years
- 77 types of processors



P2P Networks

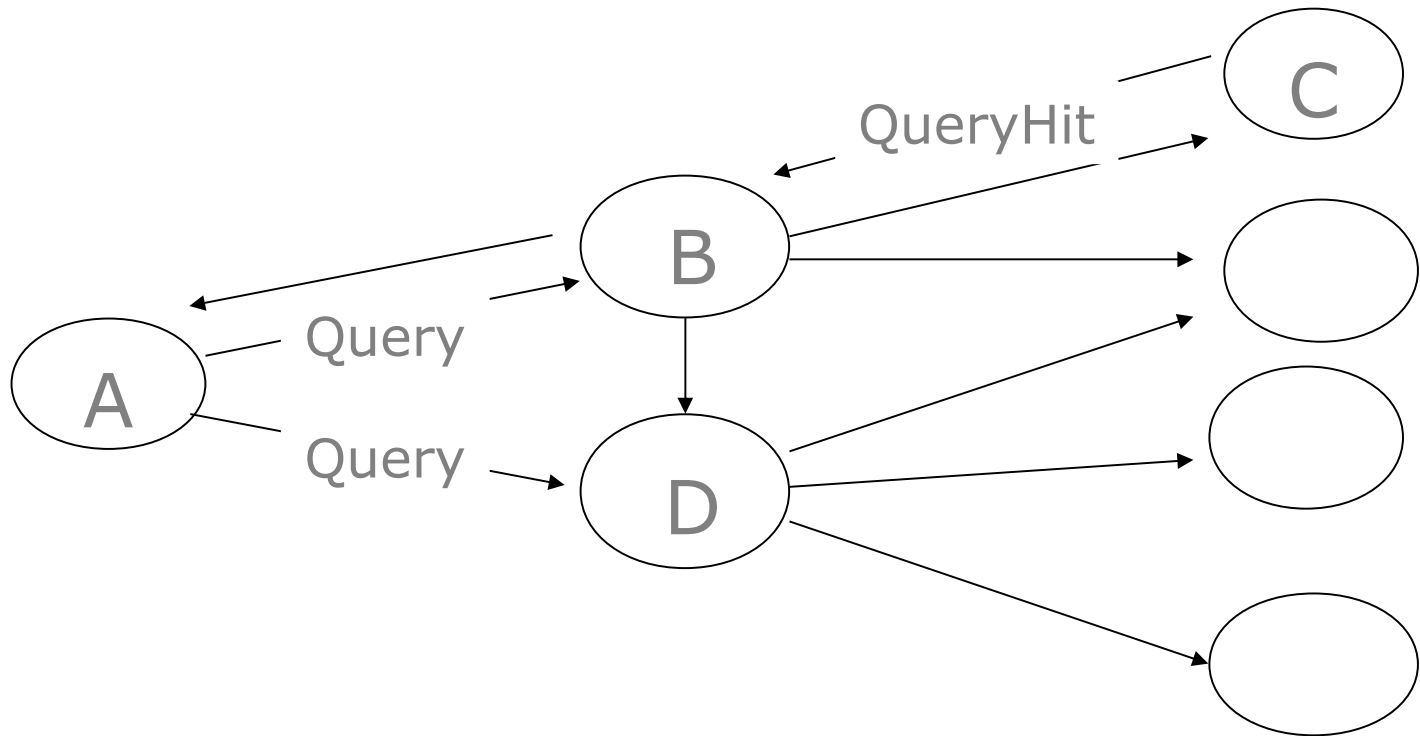
P2P Infrastructures

- unstructured (Gnutella, Kazaa)
 - The topology is determined by the users
 - Data placement arbitrary
- structured (Chord, CAN, Pastry, Tapestry, Kademlia)
 - Data is placed at specific locations (use of « hash » functions)
 - The topology of the network has specific shapes (e.g. ring, tree, grid)

Gnutella

- Protocol for data/service lookup
 - Each node is client/server
- Messages Gnutella (TimeToLive)
 - Nodes discovery PING/PONG
 - Data/Services discovery
 - Query
 - QueryHit

Gnutella



FreeNet

- Permanent Storage of data and services
 - Data identified by a key according to a hash function
 - Data crossing a node copied at the node
 - LRU politic to manage local caches
 - Stored information
 - Hash code
 - Last access time

FreeNet

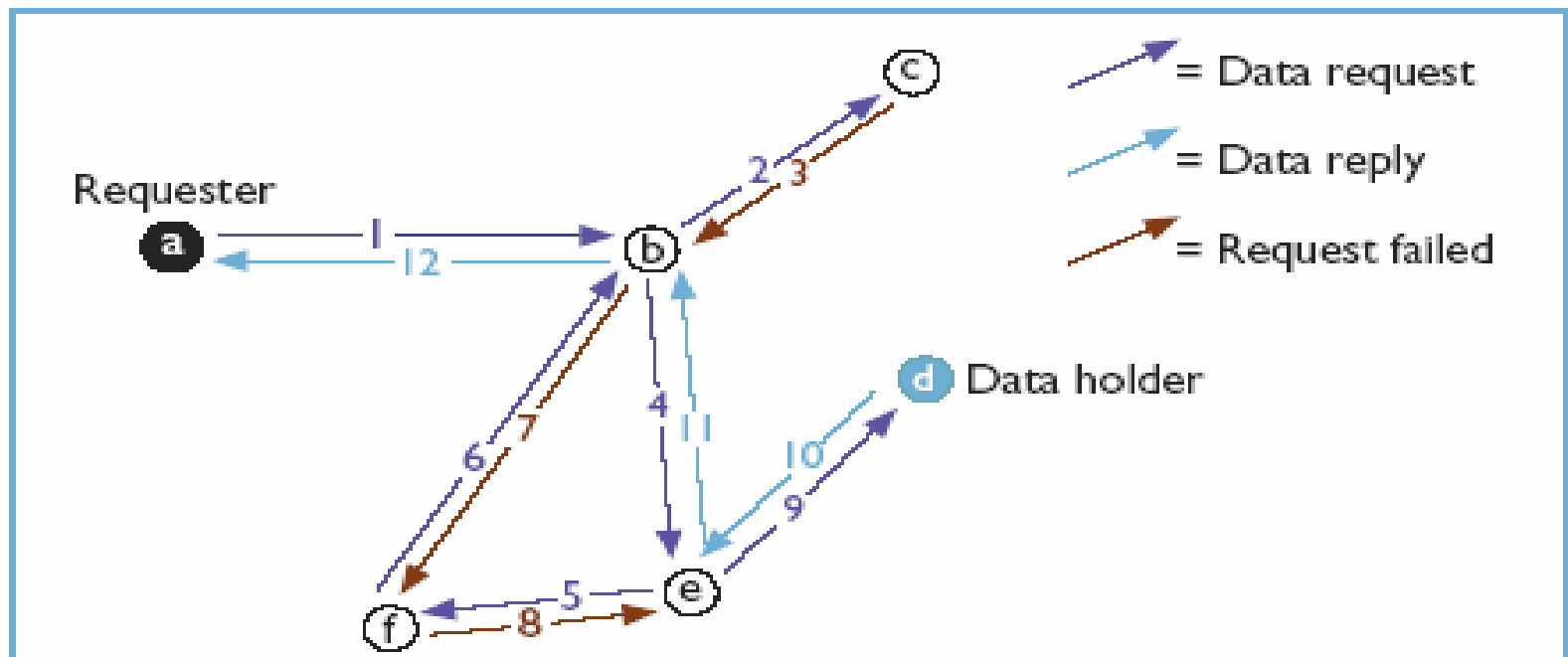


Figure 1. Typical request sequence. The request moves through the network from node to node, backing out of a dead-end (step 3) and a loop (step 7) before locating the desired file.

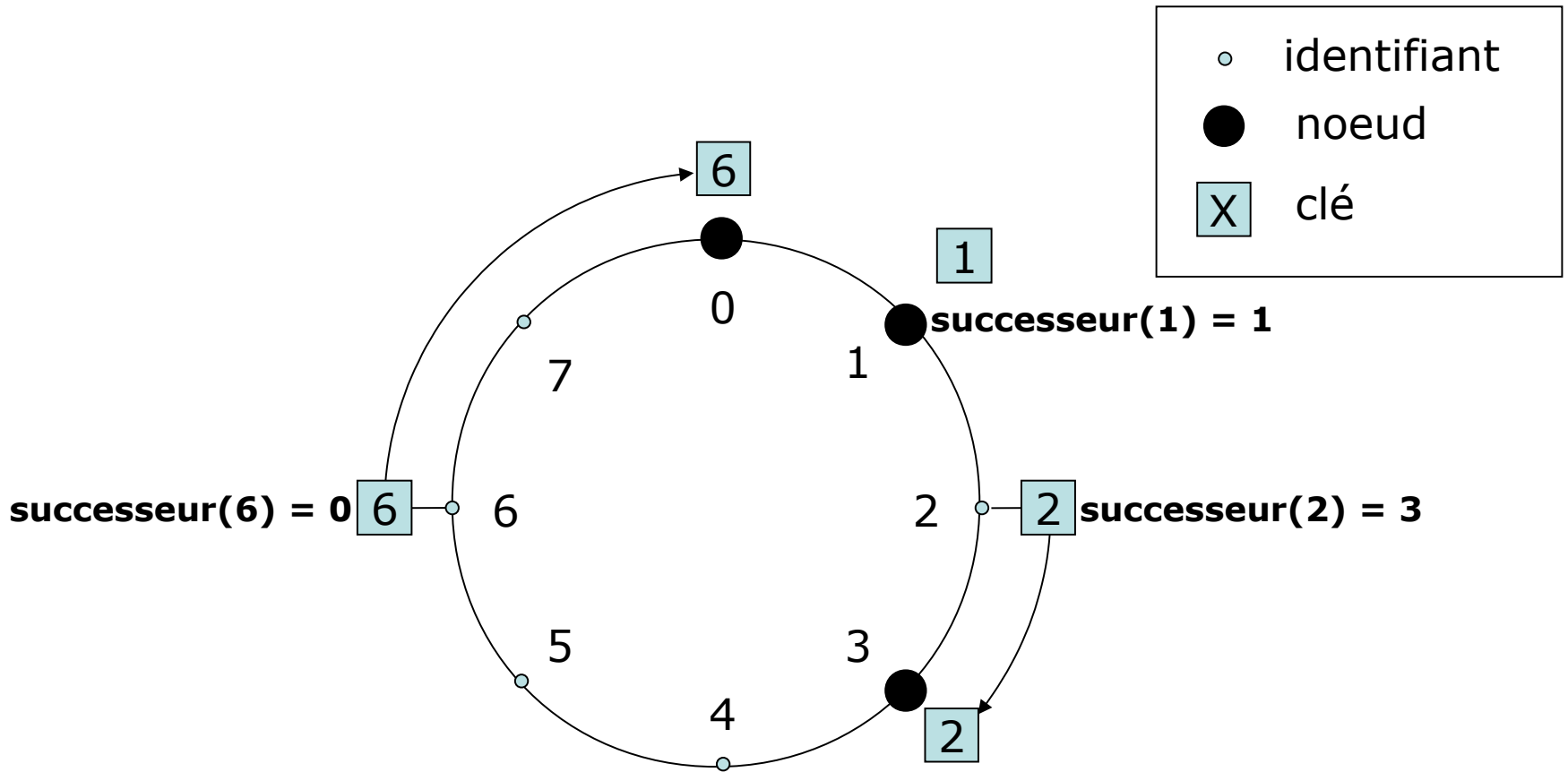
Gnutella vs. FreeNet

- Routing diffusion (flooding) vs. dynamic routing based on the keys similarities
- Zero memory of past traffic vs. Routing Tables
- Read-only vs. Read/Write
- Zero security vs. Secured System

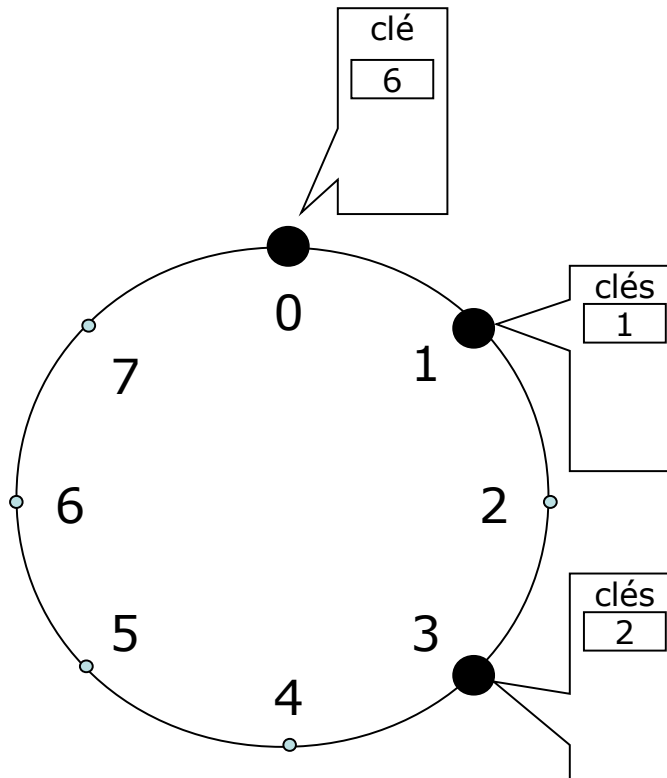
Chord

- Infrastructure for storage and routing
- Each user and data has an identifier on m bits (2^m identifiers)
 - $\text{Id user} = \text{hash}(\text{IP})$
 - $\text{Id file/data} = \text{hash}(\text{content})$
 - IDs organized in a ring topology
 - A file or data of key k stored on the first node with identifier y ($y > k \bmod 2^m$)

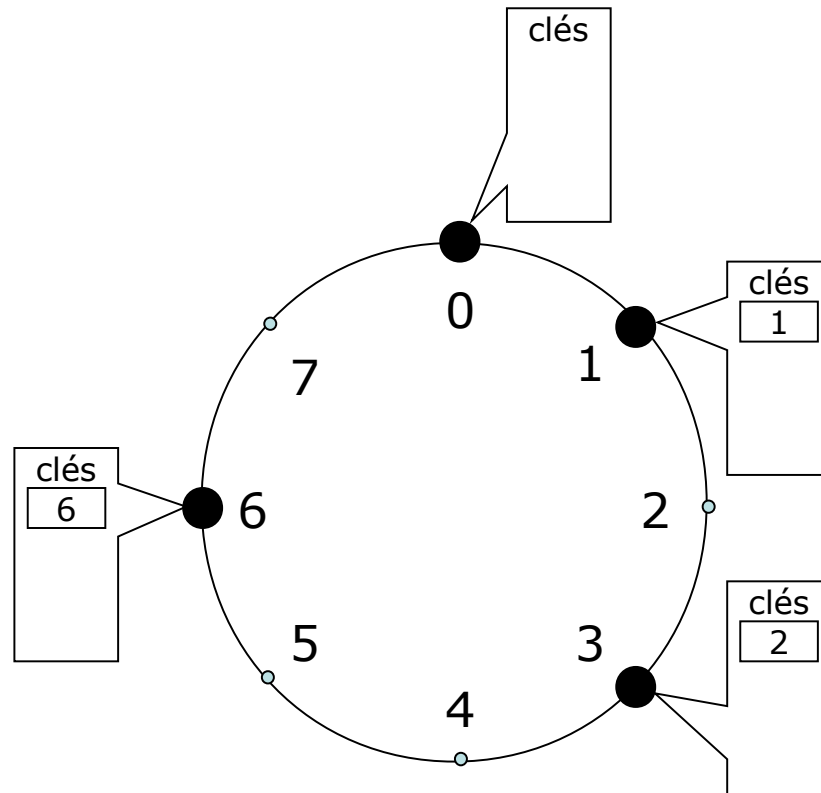
Chord



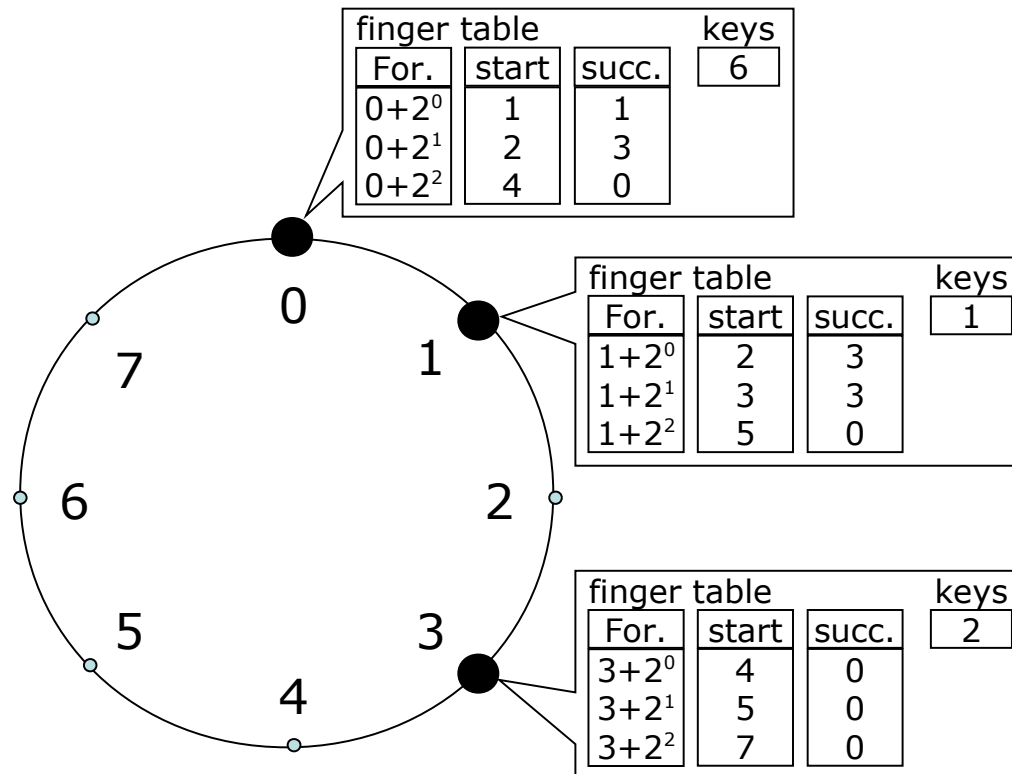
Chord



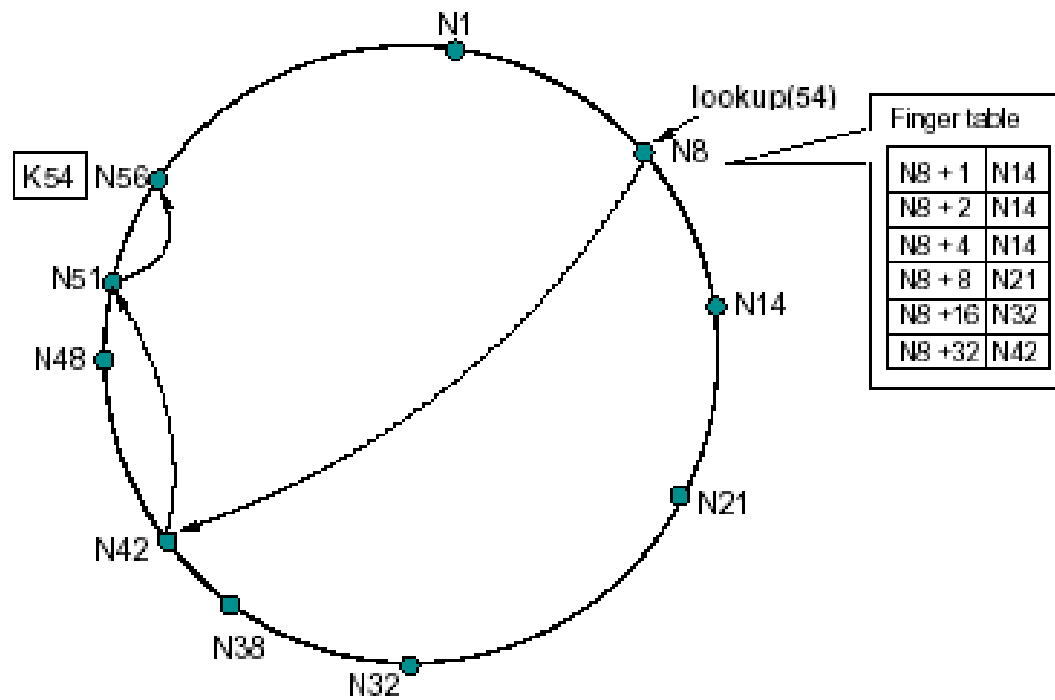
Chord



Chord with shortcuts



Lookup in Chord



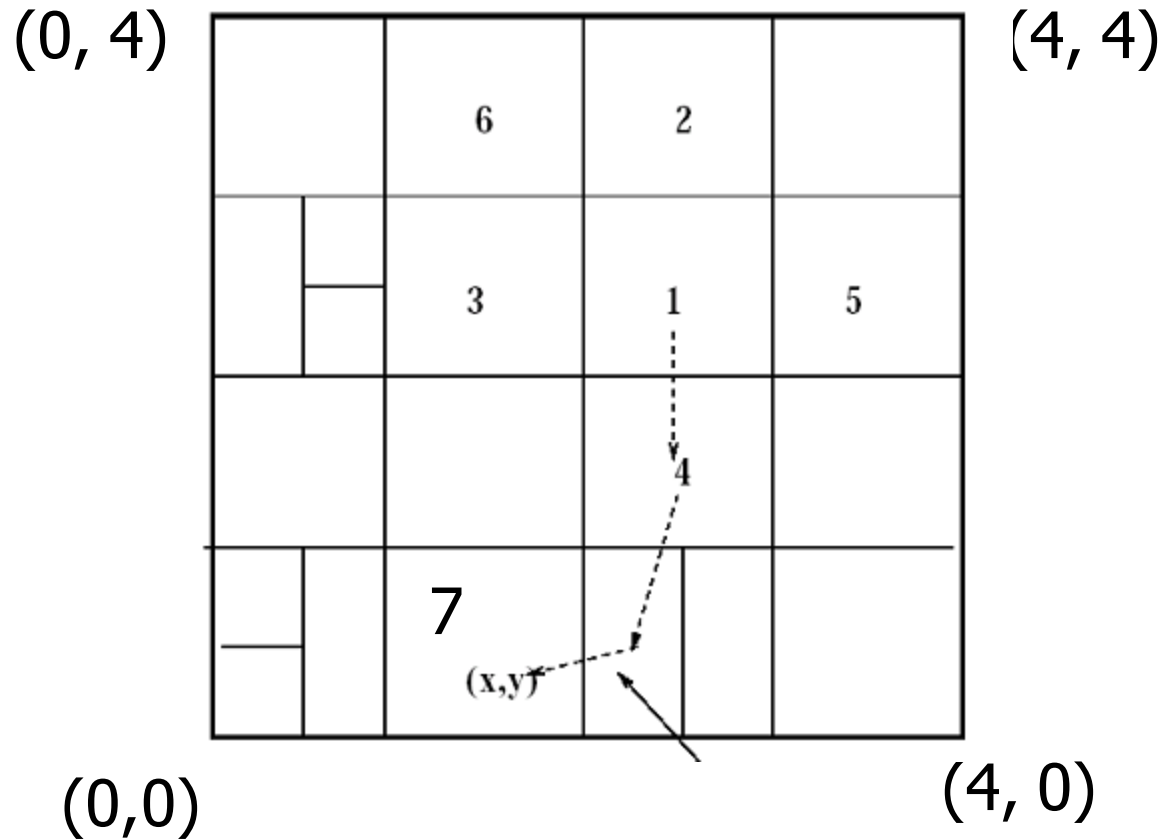
Chord

- Used memory per node $O(\log(N))$
- Lookup time for a key $O(\log(N))$
- System self-configurable and tolerant to faults and crashes

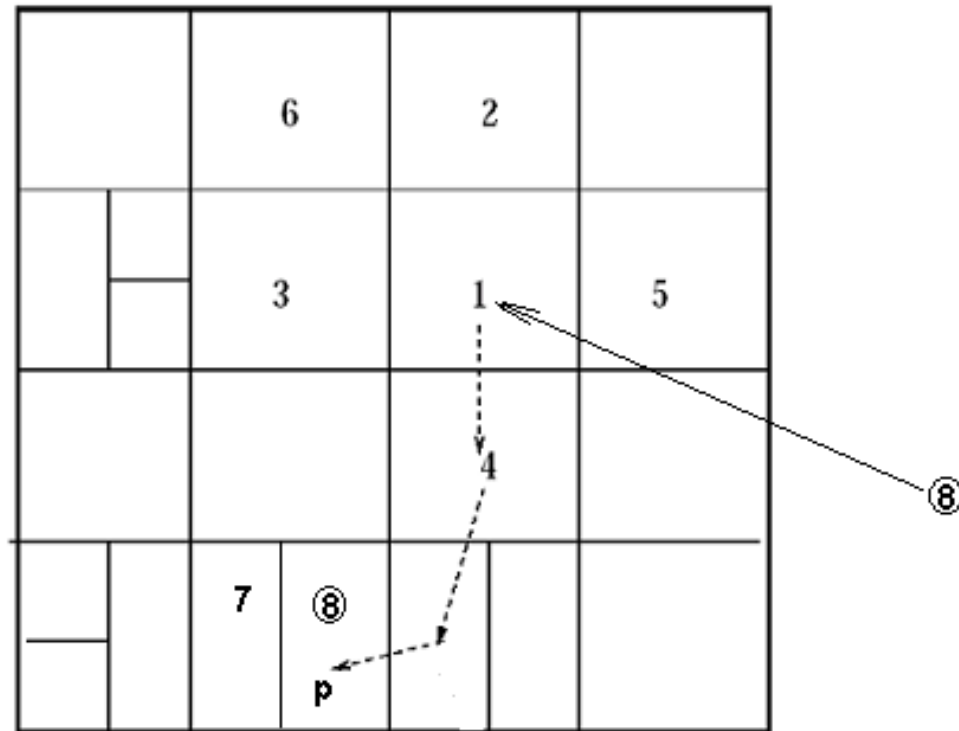
CAN

- Idea of the design :
 - 2D cartezian space
 - Each node is a owner of a virtual zone in the 2D space
 - Data stored as (key, value)
 - $\text{hash}(\text{key}) \rightarrow$ a point (x,y) in the virtual space
 - (key, value) is stored by the node owner of the zone where (x,y) is situated

CAN routing



CAN : join of a node



CAN : departure of a node

		6		2			
		3		1		5	
				4			
		7	⑧	9			
				10			

Kademlia

- nodes, files, keys are identified in 160 bits space
- every node maintains information about
ids (e.g. files, keys) “close to itself”
 - The closeness metric between two objects a and b : $D(a, b) = a \text{ XOR } b$

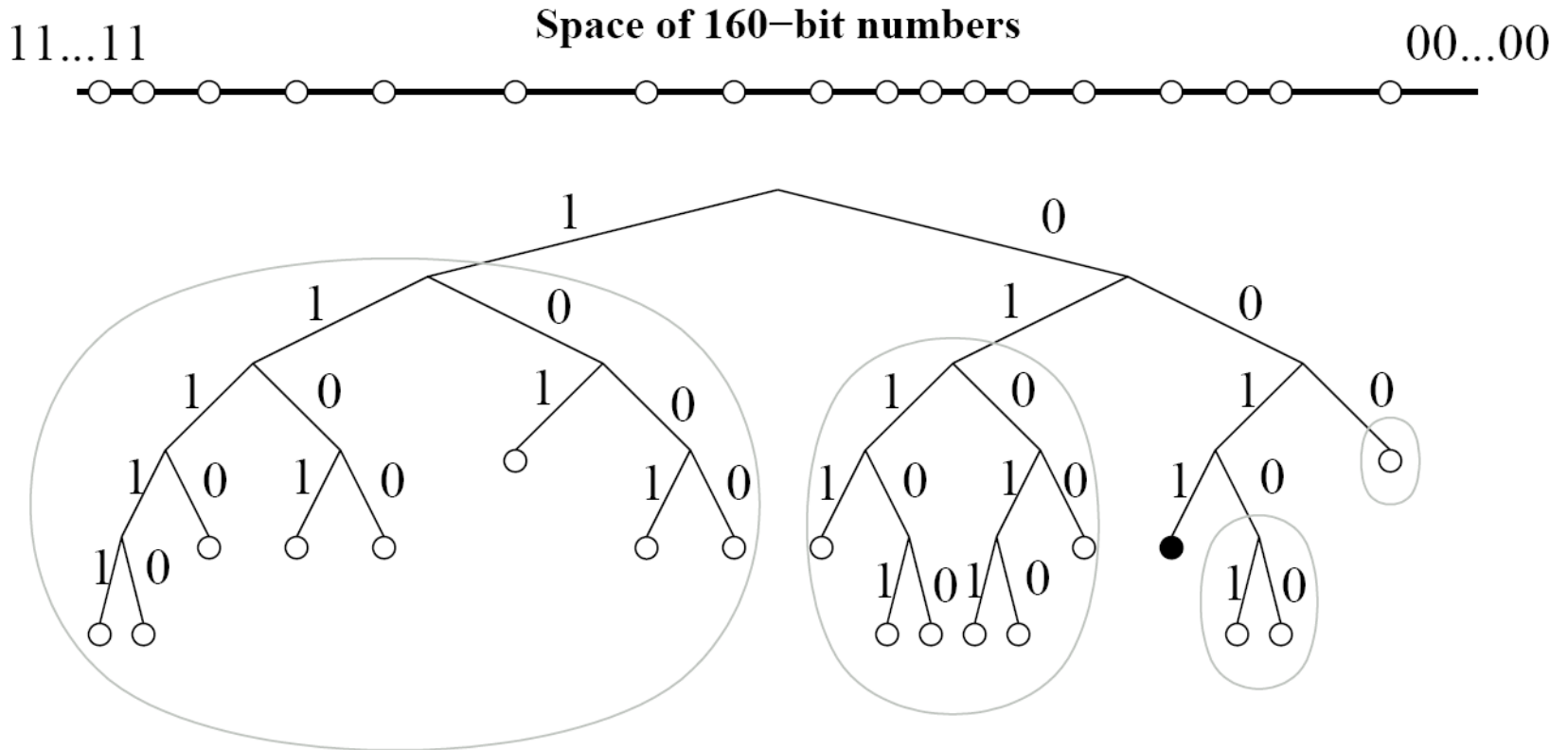
Kademlia

- XOR rules :
 - Rule 1 : both bits 1 then XOR'ed bit is 0
 - Rule 2 : both bits are 0 then XOR'ed bit is 0
 - Rule 3 : one bit is 0 and one bit is 1 XOR'ed bit is 1
- Example :
 - Object a = 1001
 - Object b = 1010
 - $a \text{ XOR } b = 0011$

Kademlia Binary Tree

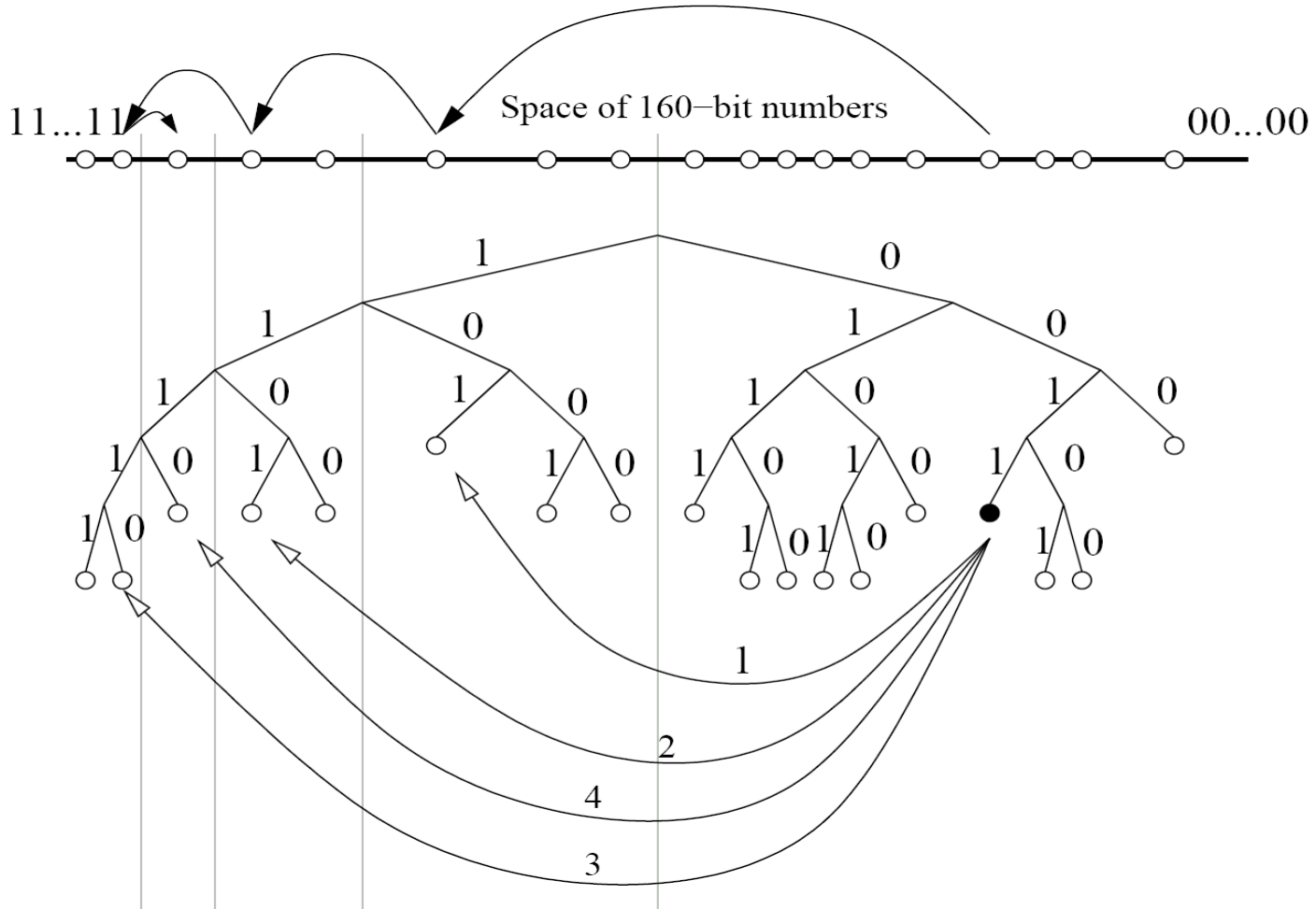
- nodes represented as a binary tree
- every node Kademlia divides the binary tree into subtrees
 - The highest subtree – half of the binary tree that does not contain the node
 - The next subtree - half of the rest that does not contain the node
 - etc
- Kademlia protocol ensures that every node knows at least one node in each of its subtrees

Kademlia Binary Tree



Subtrees for node 0011.....

Kademlia Search



node id 0011..... search id 1110.....

Kademlia nodes

- Each node maintains for each value $0 \leq i < 160$ a list (a.k.a. k-bucket) of $\langle \text{IP}, \text{UDPport}, \text{NodeID} \rangle$ for nodes of distance between 2^i and $2^{(i+1)}$ from itself
- k-buckets are sorted such that the least recently seen is at the head
- When a node receives a message from another node it updates the corresponding k-bucket by putting the sender at the tail (rear) of the list

Kademlia messages

- PING: tests whether a node is online
- STORE: instructs a node to store a <key, value>
- FIND_NODE(ID): a recipient returns k closest nodes with respect to target ID from its current buckets.
- FIND_VALUE: similar to FIND_NODE, unless the recipient received a STORE for that key, it just returns the stored value.

Kademlia Lookup

- the process locates the k closest nodes to some given ID
- lookup initiator starts by picking α nodes from its closest non-empty k -bucket*
- the initiator then sends parallel, asynchronous FIND_NODE to the α nodes it has chosen

* α , concurrency parameter, is usually set to 3

Kademlia Lookup

- Recursively the initiator resends the message FIND_NODE to nodes it has learned about from previous round.
- If a round fails to return a node closer than the closest already seen, the initiator resends the FIND_NODE to all of the k closest nodes it has not already queried.
- The lookup could terminate when the initiator has queried and gotten responses from the k closest nodes it has seen.

Kademlia Keys Store

- To store a $\langle \text{key}, \text{value} \rangle$ pair, a participant locates the k closest nodes to the key and sends them STORE messages.
- each node re-publishes $\langle \text{key}, \text{value} \rangle$ to keep them alive (every 24h for file sharing applications)

Kademlia join

- To join a node **u** has to know a node **w** already in the network
- **u** inserts **w** in the appropriate k-bucket
- **u** starts a lookup for its own id hence **u** populates its own k-buckets and populates the buckets of the queried nodes

P2P Overlays & Blockchains

- unstructured P2P overlay with gossip communication
 - Bitcoin
 - Algorand
- DHT-based
 - Ethereum – Kademlia

P2P – data replication

- FreeNet – data are copied by the nodes participants at their routing
- MojoNation – data highly demanded is disseminated in the network by a server
- CAN (multi-dimensionel) – a data can have a key per dimension
- Kademlia – data is replicated in nodes close to the data id

P2P security issues



"On the Internet, nobody knows you're a dog."

P2P security

- Freenet
 - avoids the direct communication between the data sender and receiver
 - Use of random TTL
- CAN/Chort dissociates the data owner from the node that stores data

P2P Security

- Data verification
 - cryptographic keys (CFS, Past)
- Data diffusion
 - Files are decomposed in n blocks such that m blocks ($m < n$) are sufficient to reconstruct the original file (Publius, Mnemosyne, FreeHaven)

P2P – security issues

- « free-riding » et collusion
 - One or more users take advantage of the system without sharing
 - Problem: system collapses
 - Solutions :
 - Use techniques for incite people to participat : discover of the system ressources direct proportional with the participation of nodes
 - Micro-payements (MojoNation)
 - Peers survelience

P2P – security issues

- « sybil attack » [Douceur 2002]
 - A user joins the system with several identities
 - Solution (not realistic) : unique identification of the ressources of a node